

Cart Recommendation System

Case Study Executive Summary – Module 01

Executive summary

This project delivers a production cart recommender: given products already in a shopper's basket, the service returns a ranked list of items to suggest next. The system is not a notebook experiment. It is a trained model behind a public HTTP API, with business rules so customers are not sent to products that cannot be sold, and with an operations plan so the site does not need an engineer every morning.

Live endpoint (grader contract): POST <http://RecommenderLoadBalancer-1766114668.us-east-1.elb.amazonaws.com>

Request: `{"cart":["240","200","277","78"],"top_n":10}`

Response: JSON arrays **recommendations** (string IDs) and **scores**. Health check: GET `/health` returns `{"status":"ok"}`. Measured latency is well under the 500 ms SLO (typically under 5 ms).

Business problem

A co-purchase model is only useful if the website can call it reliably and if recommendations are sellable. Three operational failures matter as much as accuracy:

1. **Drift and catalog growth.** New SKUs have no sales history, so they never appear in co-purchase scores (cold start and popularity bias).
2. **Dead product pages.** Ranking items that are discontinued or out of stock destroys trust.
3. **Cost of data.** A full cart-history extract takes about four hours and costs \$1.21–\$4.53. Retraining every night is not justified.

Approach

Scoring. Several models were compared on HitRate@10 (a hit if any of the true remaining cart items appear in the top 10). The served model is a **temporal co-purchase ranker** (180-day exponential half-life): recent carts count more than old ones. An LSTM next-item network was trained on the same leak-free January 2016 split and **lost** to temporal weighting, so it is not deployed.

Serving. The model is serialized with joblib, packaged in Docker, and run on AWS ECS Fargate behind an Application Load Balancer (public port 8000 to container 8080). Inventory and merchandising are **not** baked into the weights. Nightly JSON files drive a serving layer that (a) drops discontinued and out-of-stock SKUs, (b) pins up to three promotional / new-arrival items, and (c) injects one randomized in-stock new SKU (balanced exploration).

Key results

Evaluation	Model	HitRate@10
Train through 2015, test January 2016 (281 cases)	Unweighted co-purchase	0.3986 (112/281)
Same split	Temporal, 180-day half-life	0.4733 (133/281)
Same split	LSTM next-item	0.4270 (120/281)
<code>cart_export_17_10</code> , test November 2017 (1,996 cases)	Unweighted co-purchase	0.5772
Same	Temporal, 180-day	0.5862 (1170/1996)
<code>cart_export_19_05</code> , November 2017 holdout	Temporal (promoted artifact)	0.6042 (1206/1996)

Phase 2 (same co-purchase logic, January vs December 2016) did **not** show degradation over 11 months (0.53 to 0.56). Relationships among known products were stable. What *does* break the model is **new products**. Distinct SKUs grew from 253 through 2015 to 296 through 2016 (+17%) to 327 through 2017 (+10.5%): 31 first-seen SKUs in 2017. **733 of 1,996** November 2017 test cases have a 2017-new SKU in the answer. A model trained only through 2016 cannot rank those items at all.

Override example (grader cart [240, 200, 277, 78]): the raw model would have ranked SKUs 275, 4, and 329, which are out of stock. After rules, those IDs are removed; 388/382/381 are pinned as new arrivals; one new SKU is explored; remaining slots are sellable co-purchase neighbors (331, 172, 111, and others). No in-cart, discontinued, or out-of-stock IDs are returned.

Operating model

Cadence	Action	Human?
Nightly	Refresh discontinued, out-of-stock, new-item, and special JSON; reload rules without retraining or a warehouse extract	No
Monthly	One approved cart extract; retrain; promote only if holdout HitRate@10 is at least 0.40; keep previous joblib as backup	Yes – extract spend
Triggered	Extra retrain if 15+ new SKUs in a week, HitRate drops 5pp or more, or a major catalog purge	Yes if a new extract is required

Nightly jobs must never request cart history. Merchandising still owns campaign pins. Automation owns inventory filtering and gated model promotion.

Recommendation

Ship the live temporal API as the production recommender. Retrain **monthly**, not nightly. Keep cold-start and stock rules in the serving layer so launches and stock-outs do not wait for a four-hour extract. Monitor stale JSON (older than 26 hours), empty new-item lists after a launch, failed HitRate gates, and load-balancer health on `GET /health`.